

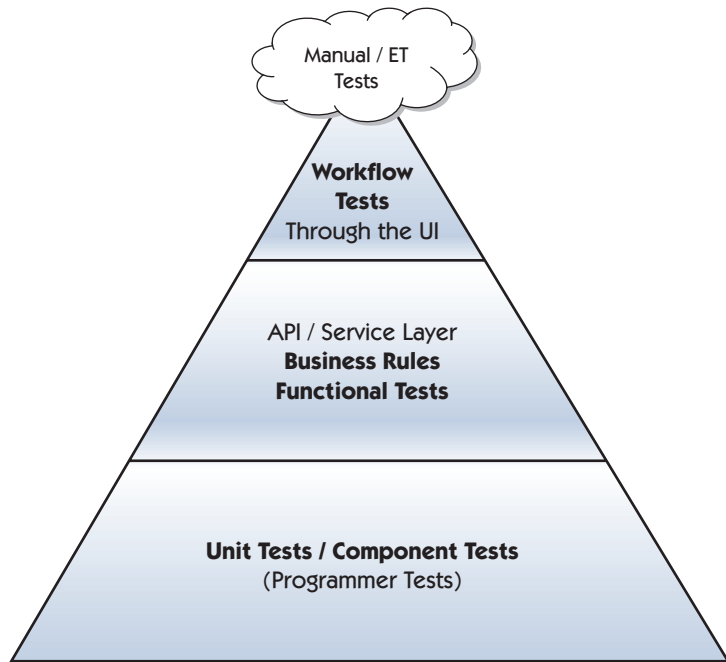


Figure 15-1 Test automation pyramid

The frameworks and tools for testing at the API/service layer and at the UI level keep improving, and they allow us to automate tests for more complex software. See more about this in Chapter 16, “Test Automation Design Patterns and Approaches.”

ALTERNATE FORMS OF THE PYRAMID

Over the years, we’ve realized that the original pyramid doesn’t fit all situations, and advances in technology have changed some of the assumptions. We’ve learned some good ways to modify the pyramid to fit various situations but still keep the original idea of where to automate.

Janet’s Story

In one organization that I visited, the team had read *Agile Testing* and told me that the pyramid was useless to them. I couldn’t understand how that could be, but I listened to why they couldn’t use it. Their problem was that the programmers really didn’t do very much unit-level testing, since they

didn't do a lot of code, and they didn't see how they could automate at the API level. They mostly gathered data from a lot of different data feeds and then manipulated it to display to the decision makers. I understood their problem and facilitated a workshop to figure out alternatives.

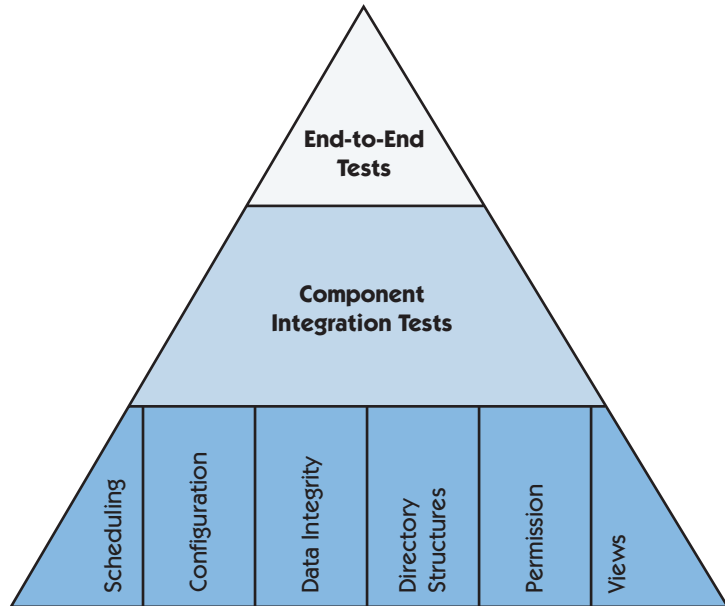


Figure 15-2 Alternative test pyramid—no unit tests

First we discussed the types of problems they were experiencing and then worked through what kinds of tests might help mitigate those problems. We then discussed when the best time to run the tests might be and who might be the best people to own them. The result is the pyramid shown in Figure 15-2. The lowest layer consists of the tests the programmers took ownership of because they wanted fast feedback. They discussed ways they could automate them and decided to experiment with several options. The middle layer comprised their component integration tests, and the slowest feedback came from their end-to-end tests.

Software products are becoming more complex, and users demand quicker response time and more intuitive interfaces. Web applications increasingly rely on JavaScript to improve the user experience, so the abstraction between business logic and the presentation layer has blurred. There are tools that allow programmers to unit test these

complexities in the UI, but extensive functional UI automation may be required in these situations. Many teams can't rely on lower-level tests to guard against regressions.

Lisa's Story

The team I joined in 2003 resolved to keep our web-based financial services application's UI as thin as possible. However, as technology and user expectations evolved, we needed to provide a richer, mistake-proof user experience. More business logic on the front end required more sophisticated UI tests. Through experimentation and time spent learning, we were able to do these in a maintainable way. The top level of our test automation pyramid expanded, but the value far outweighed the costs of the additional UI tests (see Figure 15-3).

My current team's tests are a bit more hourglass shaped, for similar reasons. Our JavaScript-based UI contains highly complex business logic. When we plan and estimate front-end stories, we include time for test-driven development at the UI level as well as for adding or updating UI regression tests. This is a great illustration of why testing and coding are integral parts of software development. Although there is usually testing to do when coding is completed, test and code can't be treated as separate phases. When we talk about planning our testing, we always intend that this be part of general story, iteration, and release planning.

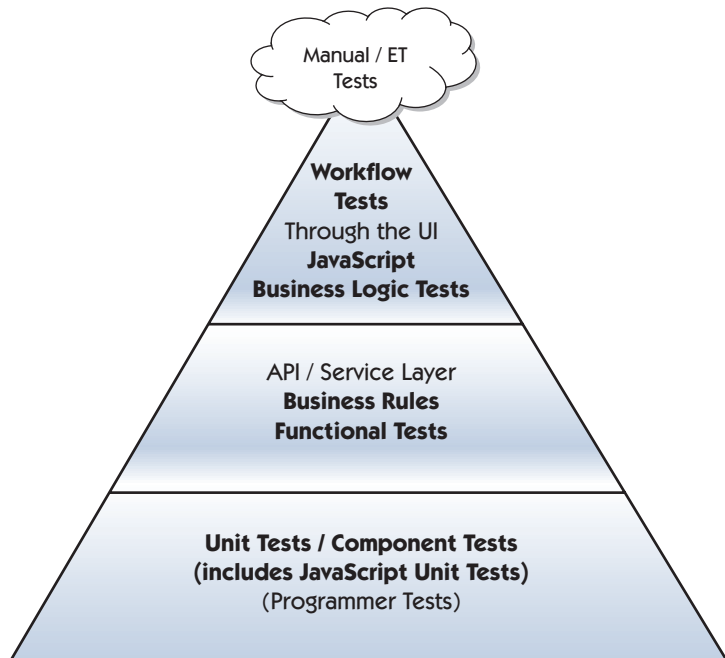


Figure 15-3 Test automation pyramid with JavaScript complexities

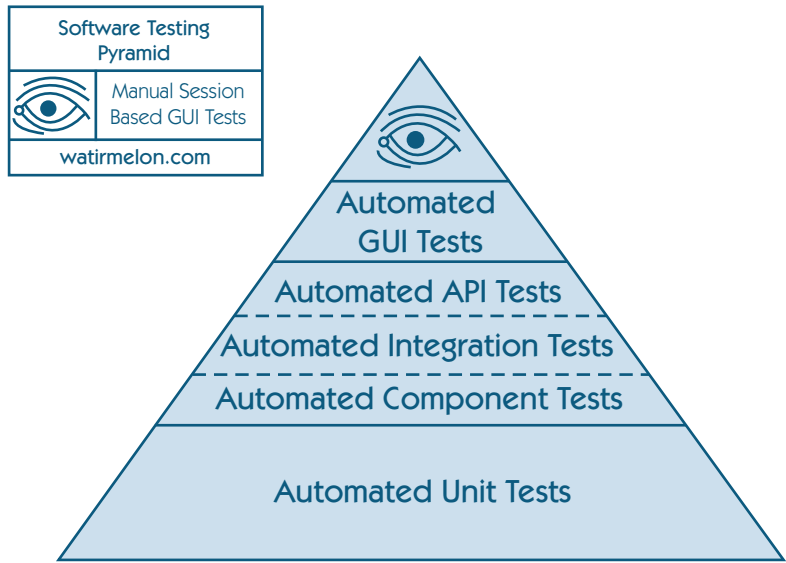


Figure 15-4 Automated test pyramid, with the Eye of Providence

Source: Scott, Alister, <http://watirmelon.com/2011/06/10/yet-another-software-testing-pyramid>, 2011. Used with permission.

We've met several teams whose situation just doesn't fit the original pyramid model. That's OK. Each team should evolve a model that fits their needs. What's important is to remember the principles behind the pyramid. The lower levels represent the fast feedback teams need as they add and update code. The goal is to involve the whole team in automating enough regression tests so that the project doesn't become mired in technical debt. Pushing tests to lower levels where they are generally easier to write and maintain helps achieve the best ROI.

Alister Scott has created a test automation pyramid that is similar to the original but puts a bit more emphasis on exploratory testing. In his opinion, exploratory or session-based testing ensures confidence in the automated tests that are being developed and run. Without it, an automated testing strategy is fundamentally flawed, which is why he includes exploratory testing as the "Eye of Providence" in his pyramid, as shown in Figure 15-4 (Scott, 2011b).

THE DANGERS OF PUTTING OFF TEST AUTOMATION

The test automation pyramid is a model intended to guide teams in getting the most value from their test automation for the minimum

investment. Matt Barcomb used the pyramid as the basis for a fable about what happens when there's no whole-team effort to implement adequate test automation, especially at the lowest level.

In the Legend of the Test Automation Volcano (see Figure 15-5), a development team lives in a little oasis at the side of a test automation pyramid. The programmers lounge under the palm trees, enjoying umbrella drinks. They're too busy doing what they think they enjoy most to actually automate any regression tests for their software product. Meanwhile, sweaty testers labor in the bowels of what is not an automation pyramid but a volcano, frantically doing manual regression testing and falling further and further behind. The tests within the volcano that would normally cool it off are missing. There's no time to devote to that exploratory testing sun at the top. Technical debt accumulates in the form of red-hot lava. If the development team doesn't collaborate with testers and automate sufficient regression tests, the lava keeps building



Figure 15-5 Automated Test Volcano

and heating up until the volcano erupts, burying the unfortunate programmers in their oasis.

Though Matt turns this into a hilarious story, which in his telling includes sacrificial virgins, the message is serious. Real-life teams working under the strain of ever-growing technical debt feel real pain. In the long term, insufficient automated regression test coverage and lack of time for essential activities such as exploratory testing will slow the team down and could possibly stop it in its tracks.

Fortunately, we hear of more and more teams that are able to apply the test automation pyramid model to get control over their technical debt and shorten their feedback loops. Let's look at some success stories.

How We Flipped Our Pyramid

*Here is a story from **Paul Shannon** and **Chris O'Dell**, agile quality enthusiasts from the UK whose team suffered from an inverted pyramid, about how they managed to flip it to stability.*

The first incarnation of the 7digital API was a single monolithic application that started small and was tested mostly with full end-to-end tests. With the application being so small, these tests were very useful, but as the application grew, the number of end-to-end tests grew with it. A point was soon reached when a full run of the test suite would take nearly an hour, and as you can imagine, this was extremely damaging to the feedback cycle and subsequent development speed.

Developers started to employ tactics for handling this long-running suite of tests, for example, running only the tests that were obviously relevant to the area they were working in on their local machines, and then relying on the continuous integration (CI) server to run the full suite. By the time the full suite had completed, the developer had often moved on to something else, so any failures were likely to have been overlooked.

The tests themselves were also prone to unexpected failure. Due to their end-to-end nature, they were very susceptible to changes in any part of the stack, such as a database becoming momentarily unavailable or a fluctuation in the internal network. These frequent instances of instability reduced trust in the tests, causing the developers to be even less inclined to run them.

The team realized that this situation was unsustainable. These end-to-end tests had been used to cover all scenarios, including all edge cases and all possible variations. It was a habit that had started when the application was small and had continued throughout its growth.

We decided that we needed to tackle this problem directly, but it was difficult to pin down—the scope was large, and it was impossible to see what was required without diving in. We took the approach that when programmers were working in a given area, they would review the associated tests, isolate the main “user journeys” to remain as end-to-end tests, and push the edge cases down to integration and unit tests.

This significantly extended the time for each feature or bug fix to be completed, and there were times when a change to the tests was so big that it needed to be tackled on its own. In these cases, we would write up a card—a work item—for the refactoring, label it as technical debt, and prioritize it against the rest of the backlog. This was not difficult to do as our product manager was very supportive of the work. He could see the effect these poorly performing tests were having on our ability to implement new features.

We also had a small whiteboard in the team area where we recorded some refactoring ideas and goals, such as “Refactor any web forms to use an MVP pattern to allow for testing of the Presenters.” This kept us focused because we were doing refactoring as part of other work.

The change was gradual and in many cases very frustrating. The state of the code base became far worse before it got better due to the transition period when different approaches were in use. The overall code base was confusing to look at, and there was a temptation to give up and start again, but we persevered. The chart in Figure 15-6 shows the huge increase in time taken to complete work items when we started to tackle the problem. You can clearly see at point A when we started this initiative as the cycle time increased. The positive effect on cycle time can be seen later at point B, when the hard work began to pay off and cycle time was reduced, showing an overall downward trend.

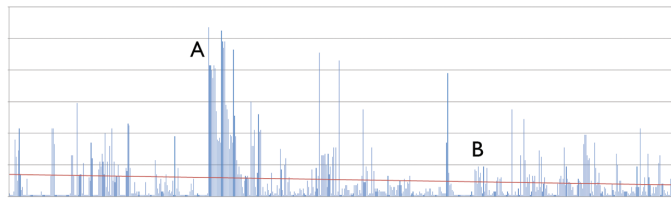


Figure 15-6 Cycle time of work item completion

The effort paid off and the end-to-end tests' run time came down from nearly an hour to under five minutes with more than quadruple the number of unit tests. Developers were able to run the unit tests on their local machines after each change, remaining confident that virtually all scenarios were covered. The full suite of tests is now run only before pushing changes back to the main branch.

USING THE PYRAMID TO SHOW DIFFERENT DIMENSIONS

We call it a pyramid, but really it is a triangle. We prefer simple models and to use more than one when applicable, such as the simple pyramid and the agile testing quadrants. We find that the more complex a model is, the harder it is to understand. People have been adding dimensions to the pyramid, some with more success than others. The next story is one of those successes.

The Test Automation Pyramid—Expanded

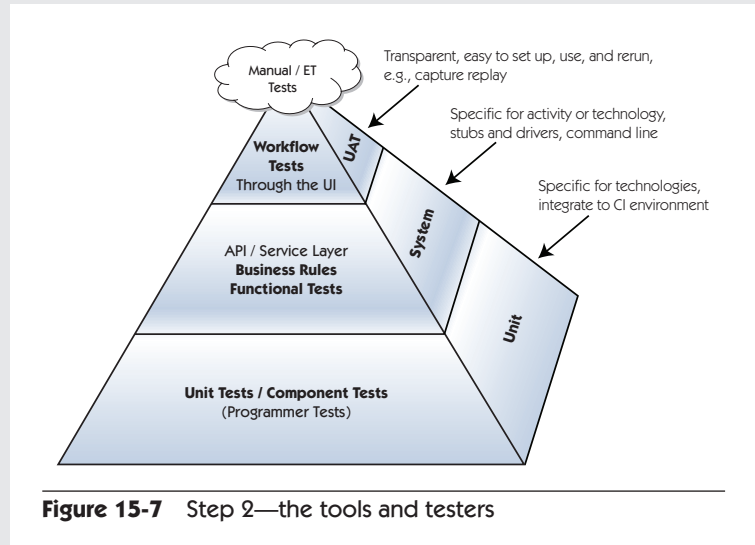
Sharon Robson gives the test automation pyramid some new dimensions, reflecting the various dimensions quality teams need to build into their software products.

I interpret the agile testing pyramid as a beginning to a discussion about test strategy in an agile team. I really like the pyramid analogy because a pyramid is a self-supporting structure that cannot exist without each of its sides. Each has a different focus (or direction), but each ties into the whole unit. I go through four steps to explain the pyramid to teams.

Step 1 is a basic explanation of the pyramid (see Figure 15-1)—what each layer means and what each layer is designed to test. This aids in appropriate test design for the team members.

Step 2 is adding the next (right) side to the pyramid (see Figure 15-7), which encompasses the tools that we can use to execute these tests. Each tool is chosen for the test type and the test executor. For

example, at the functional and business rules test layer, we are looking at system testing and making sure the fundamental functionality of each of the architectural elements works both individually and together. At the UI level we are looking for user acceptance testing. This means that we need to have the right type of tool for the executors of the testing at that level to use.



This expansion allows people to understand that there is not one tool that will do everything, and that when we are automating we need to match the tool to the tester as well as the testing. This model allows clarity about how the tool types and tool users change depending on where inside the application or architecture we are focusing.

Step 3 is to extend the pyramid to show three sides (see Figure 15-8), with the overlay of test types or system attributes. These allow us to be sure that we consider all the aspects of the solution that need to be tested.

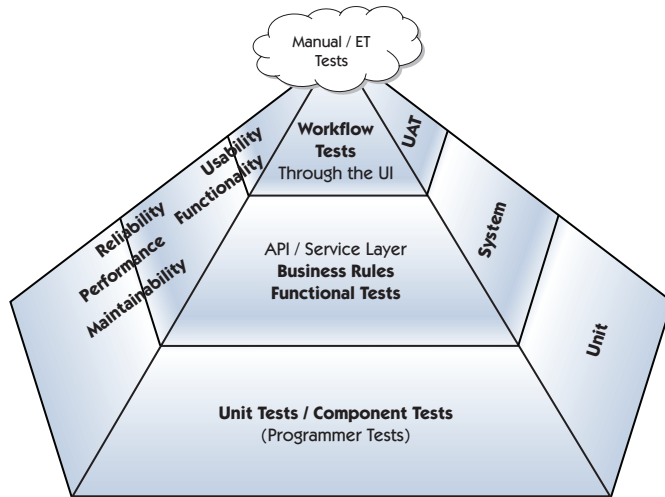


Figure 15-8 Step 3—the test types

After these three steps we can see who is going to focus on what type of testing—for example, functional versus nonfunctional—and why these tests should be run at the various levels. This then allows us to check and see if the tool selection and the tests have been designed for coverage of the system attribute we are trying to evaluate.

Once a test attribute is assigned to the test level, people in the team begin to understand how much testing may be involved, and we start to talk about data sets, reuse of data, tests and test design, as well as test design for specific attributes. We also encourage technical team members to share their knowledge of how to test the more technical (architecture and design) attributes at the higher levels. Nontechnical team members start to understand why test automation and test design are important for coverage and for building the testing resources for regression.

Visually it is pretty hard to represent that fourth side of the pyramid, so I leave the regression tests as lines running down the third side,

indicating that the regression can be on any system attribute that has been considered as part of the testing (see Figure 15-9).

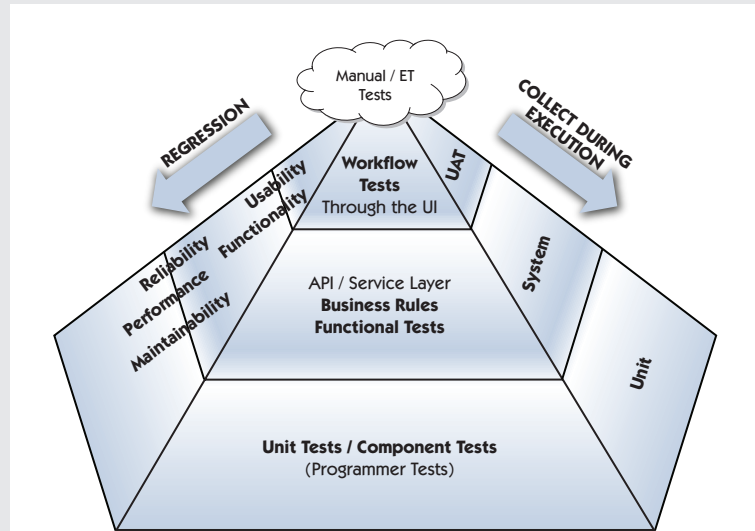


Figure 15-9 Step 4—regression tests

I use this four-step process when I am talking about automation in general, not just in the agile framework. It is very useful when considering testing done by the whole team, identifying the consistent and reusable test and data sets that will be required, and explaining the need for different types and levels of testing to prove that each of the system components, elements, and attributes hangs together in a solution that “works” in a way that we expect.

Adapt the test automation pyramid to fit the needs of your team, your technology stack, and your product. Use it to automate tests at the optimum level for fast feedback on regression failures. The pyramid model helps your team remember to push tests down to the lowest level that makes sense, maximize its return on the investment in automation, and find ways to build in a variety of quality characteristics.

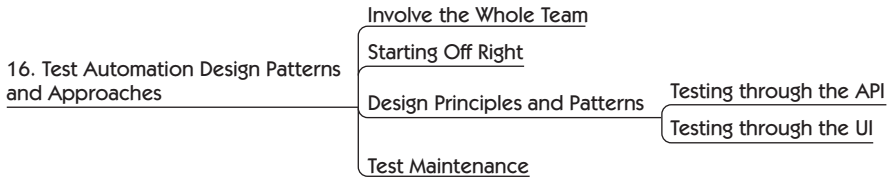
SUMMARY

The original test automation pyramid is useful, but it's not perfect for every situation. We've shared some ways we and other agile testing practitioners have adapted and expanded the pyramid to represent automated test strategies in a visual model. These different models include

- The tried-and-true original test automation pyramid
- Test pyramid with no unit tests but other programmer tests instead
- Pyramid showing where JavaScript tests fit
- Alister Scott's test pyramid with the Eye of Providence representing exploratory testing
- The lessons learned in the Fable of the Automated Test Volcano from Matt Barcomb
- Sharon Robson's expanded test pyramid representing multiple quality dimensions, tools, and types of testing

This page intentionally left blank

TEST AUTOMATION DESIGN PATTERNS AND APPROACHES



Since writing our first book, we’ve met more and more teams that have achieved their test automation goals. This is great news. However, it is still hard to learn how to automate tests and probably always will be. Current automation frameworks enable teams to automate tests at all levels in a syntax that business experts can understand. Yet difficulties such as sporadic test failures due to timing issues in user interface (UI) tests continue to plague us. The test automation “hump of pain,” as described in *Agile Testing* (p. 266), still looms as an imposing barrier to teams that are new to test automation.

Teams that adopted agile found existing functional test automation frameworks and drivers wanting, so they created their own. Then, many of them open-sourced those tools. Today we have many sophisticated functional test tools designed for use in agile projects. Testers and programmers doing acceptance-test-driven development (ATDD)/specification by example (SBE)/behavior-driven development (BDD) no longer have cause to envy the cool tool sets available to test-driven development (TDD) practitioners.

We need to evolve our tests and processes to take advantage of the tool changes. Our products will change, which likely means changes to existing test structures. Adam Knight (Knight, 2014) suggests asking ourselves some questions, such as, “Is our test structure extensible if we

need to support new interfaces?” Over the years, and through the ability to develop and extend the test harnesses, Adam has been able to add in support for parallel execution and include iterative scaling of test execution and multiple server runs. We hear about experiences like Adam’s more frequently with each year that passes. Each team must be prepared to take on similar challenges.

INVOLVE THE WHOLE TEAM

The whole-team approach to testing and quality is possibly most critical when it comes to automating tests. Automated tests are code. Not only do they protect us against regression failures, but they help us to document our production code, telling us exactly what our system does. As we’ve mentioned before, they deserve the same care and feeding as our production code.

When testers own test automation, they must spend large portions of time writing test scripts for stories in the current iteration, investigating test failures, and maintaining the existing automated tests so they continue to work as the production code is updated. There’s often little time left for crucial activities such as exploratory testing. Programmers who aren’t automating functional tests have no incentive to create testable code because they don’t feel the pain of code that’s not automation friendly.

When the whole team is involved in test automation, the programmers recognize how they can make their code testable. For example, they can design the code with different layers, each of which can be tested independently. For a web-based application, simply using unique identifiers for HTML elements rather than using dynamic naming makes automating UI tests easier.

It is important for the whole team to own, and see the value of, automation, so that the work can be shared where it makes the most sense. It makes sense for the people who are best at writing code to write the test code. We do know people who self-identify as testers who are also excellent coders and do a great job of designing automated tests. However, on most teams, the people with the most coding experience are the programmers, and their skills can be used for the test code. Creating tests and writing the code to make them run are two different skill sets.

Testers are good at knowing which tests to specify and which tests to change if existing functionality is being changed. Collaborating with each other to implement test automation makes sense (see the section later in this chapter on “Testing through the UI”). As we said in *Agile Testing* (pp. 300–01), team members in other roles, such as system administrators and database administrators, also contribute to good automation solutions.

STARTING OFF RIGHT



As more test automation frameworks and drivers dazzle us, it’s tempting to go the, “Oooh! Shiny! And it would look good on my résumé!” route. However, as Markus Gärtner advises (Gärtner, 2012), each team must first decide what their tests should look like. This takes lots of thought and experimentation. You need to answer many questions, such as, “Who needs to be able to read the tests? Who will specify them? Who will automate them? Into what continuous integration (CI) tool will they be integrated? Who will maintain them?”

Liz Keogh (Keogh, 2013a) suggests that teams get certain capabilities in place before “heading down the tools path”: an eye on the big picture, the ability to question and explore, the ability to spot and embrace uncertainty, as well as having great relationships between people. Learn how to have conversations to elicit examples of desired behavior before deciding how to encapsulate them into a particular tool. This keeps the focus on collaborating with business experts and helps the team creatively find what works best for their situation.

Create a domain-specific language (DSL) that will let your team guide development with customer-facing tests. A good place to start is by getting examples from your domain experts (see Chapter 11, “Getting Examples,” for more about that). Then, and only then, look for the right tools to help you create tests that business users can read; these in turn become executable specifications. By starting with Quadrant 2 tests (business-facing tests that guide and support development), the programmers will continue to use the same language throughout their unit tests and production code. This continuity creates a common language that helps to provide living documentation—documentation that is guaranteed to be up-to-date, as long as the tests are passing. As a bonus, these tests are incorporated into test suites run by your CI process,

where they also protect your end users from regression failures. See the bibliography for Part VI, “Test Automation,” for resources on learning more about DSLs.

DESIGN PRINCIPLES AND PATTERNS

In *Agile Testing* we noted that principles of good code design apply to test code as well as to production code. Principles such as Don’t Repeat Yourself (DRY) help avoid duplication and ensure that when something changes in the system under test (SUT), only one test component needs to be updated. The Arrange-Act-Assert pattern (Ottinger and Langr, 2009a) is commonly used in unit tests but applies to higher-level acceptance tests as well. In this pattern, you arrange the context by creating an object and setting its values, act by executing some method, and assert that the expected result was returned. The software community has

Table 16-1 Simple Rules to Live By for Automating Tests

Rule	Reason
Single purpose	Easier to debug; easier to change if business rules change
DRY (Don’t Repeat Yourself)	Ability to change tests in only one place
Use a DSL (domain-specific language)	Makes communication about the tests easier
Abstract code out of the tests	Makes the tests business readable
Setup and teardown tests	Can run the tests repeatedly
Independence	Tests can run independently and do not depend on order to run consistently
Avoid database access (if possible)	Database calls slow down the tests (note that somewhere you may need to test the access)
Tests must run green—all the time	Confidence in the tests; living documentation
Apply common test standards (including naming conventions)	Enables shared code/test ownership and common understanding of tests
Separate the test (the what) from test execution (the how)	Abstracting the what from the why can allow the layers to evolve separately; you can add more examples to the human-readable specification (the test), or you can change the underlying automation without affecting the business rules

continued to evolve design principles and patterns that reduce the cost of writing and maintaining automated test scripts.

Whether you are automating at the unit, API, or UI level, look for ways to improve your test design to keep long-term maintenance costs to a minimum while getting fast and useful feedback. Simple steps such as documenting your test design patterns for your team or development organization can help ensure consistency and maintainability, as well enable others to understand the structure of the tests.

Table 16-1 shows some basic design rules we think are important to keep tests maintainable. It is by no means exhaustive, but it can give you a good start on experimenting to see what works best for your team.

Testing through the API (at the Service Level)

When you start out to automate business-facing tests that guide development, spend time with the stakeholders and the delivery team deciding how you want the tests to look. The tests should be useful and understandable to all who need to use them. Remember that these tests will provide valuable living documentation about how the system behaves, if they continue to pass when changes are made.

Figure 16-1 shows how API-level testing frameworks generally work. The diagram doesn't reflect the overhead needed to create test libraries

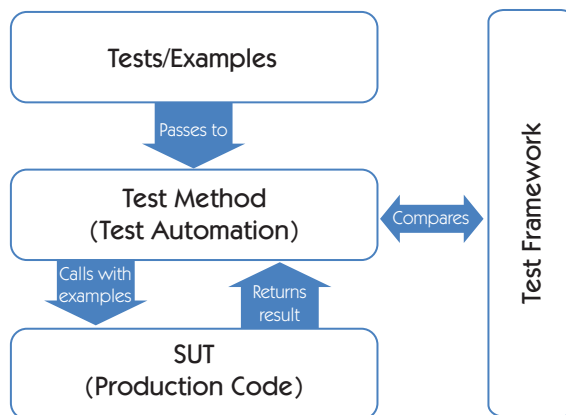


Figure 16-1 API test structure

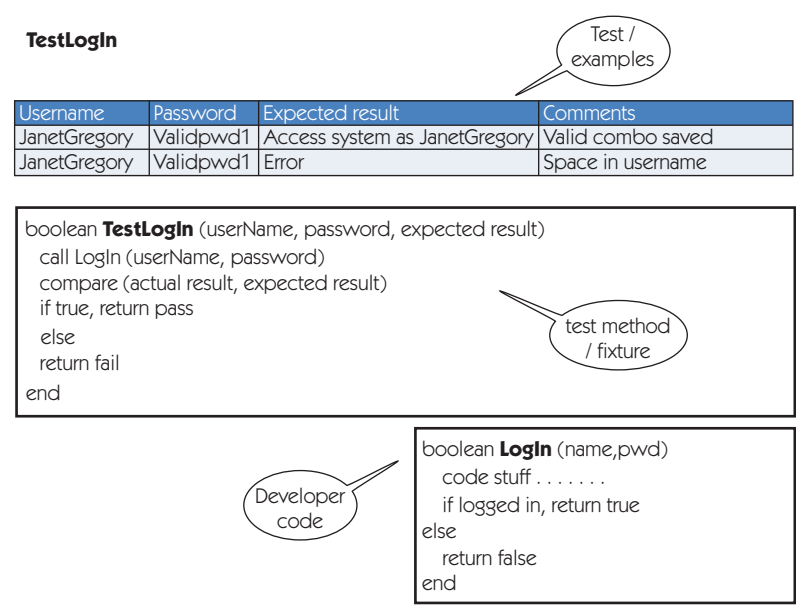


Figure 16-2 API test example

and abstractions. Rather, we want to draw attention to the magic in the middle piece—the “glue,” or test method. When testers and programmers collaborate to determine what the tests should look like, amazing things happen to enhance the shared understanding of the story. Once you have that, writing the automation code is a smaller effort.

Figure 16-2 is an example of a test for a simple login. There are two tests: one for the happy path, a valid username and password; and one with an invalid username. The input data that is passed in the first test to the TestLogin test method would be <JanetGregory, Validpwd1>. It is then passed through as input variables to the production code. The expected result <Access System as Janet Gregory> is what will be compared to the actual results coming back from the production code via the test framework. Of course, how the data is passed back and forth is something testers and coders must discuss. This collaboration on what is being passed, and how to represent it, is where the magic happens.

Chapter 11, “Getting Examples,” has more on testing below the UI and discusses how to guide development with examples. There are multiple patterns that can be used to work with this structure. One example is the Ports and Adapters pattern (Cockburn, 2005) discussed in *Agile Testing* (p. 112).

Testing through the UI

In recent years, the agile community has come up with more effective patterns and practices for designing automated tests that deliver a great ROI. This is true for all levels of automation, including the regression tests that run through the application’s UI.

Automating tests that exercise the application’s UI continues to pose the most difficult challenges. Many teams have invested time and money to automate UI tests, only to find the long-term maintenance cost overwhelming. Over the years, better approaches have evolved. Gojko Adzic (Adzic, 2010b) recommends thinking about UI automation at three levels:

- **Business rule or functionality level:** what the test is demonstrating or exercising—for example, get free shipping within the continental United States with an order totaling a certain amount
- **UI workflow level:** what high-level activities the user has to do in the UI to exercise the functionality being tested—for example, create an order whose total amount qualifies for free shipping
- **Technical activity:** the technical steps to exercise the functionality in the test—for example, open a page, type text in input fields, click buttons

This approach can be implemented via several popular testing frameworks, using step definitions and scenarios, keywords, and, with some additions at the business level, Page Objects. Alister Scott also shares his experience with the three-level split approach, with examples of executable specifications and living documentation using Cucumber (Scott, 2011a).

Page Objects, and page resources for non-object-oriented frameworks, can be used to encapsulate all the things that are testable on each page of a UI. The Page Object (see Figure 16-3) includes all the functionality to

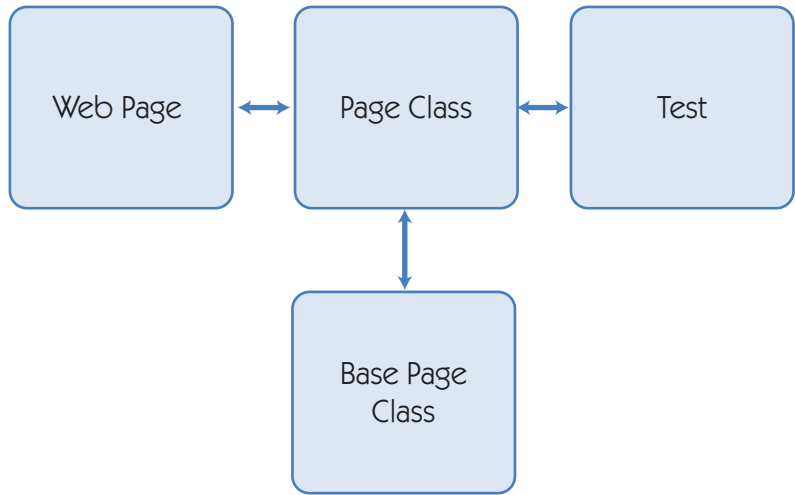


Figure 16-3 Page Object pattern

interact with the SUT via third-party test libraries such as Selenium. It is most applicable where pages and activities are well aligned. When most activities span multiple screens or pages, or where a single screen does multiple activities, the Page Object may not be a good fit, and that may lead to maintenance problems.

Jeff “Cheezy” Morgan created an open-source Page Object Ruby Gem for testing browser-based applications (see the “Tools” section of the bibliography for the link), and there are other implementations for different programming languages. It can be used in testing web applications, desktop applications, mobile applications, and even mainframes. See Chapter 20, “Agile Testing for Mobile and Embedded Systems,” for Cheezy’s story of how he automates testing mobile apps.

Cheezy explains how the Page Object pattern can be used with the three-layered approach (Morgan, 2014):

PageObject is a great pattern for building an abstraction over the system under test. You still need to provide a place where you clearly express the business rules to be verified, the workflows or paths to be taken through the application to complete the behavior, and the data needed by the

application to complete the workflow. I look at these three additional parts of the test as very distinct. I use a BDD tool like Cucumber to express the business rules or behavior and other libraries to provide the navigation and tests. I find that this separation of concerns makes my test code much cleaner and easier to adapt as the application changes.

Your team should decide how you want your tests to look and then experiment with different patterns and approaches to see what works best.

Lisa's Story

Our team's product was a web application that originally had a thin client. The business logic was all on the server side, and we automated regression tests for it at the API level. Our UI test tool worked through the HTTP layer, so we sometimes had trouble with client-side events, but it worked well to cover the UI regression testing.

Later, our team came up with new front-end code that we knew would help reduce costly user mistakes, but our UI test tool couldn't "see" the event. We felt it was too risky to implement it without automated regression tests, so it was time to find a new UI test driver and framework.

The Page Object pattern provided an appealing way to create maintainable UI tests. We kicked off a series of "bake-offs" to identify the best approach that used the Page Object and enabled the testers and coders to collaborate closely. We used a set-based-development-style approach. First, we agreed on how we'd like our tests to look. We considered a `given_then_when` style, which our product owner liked. But knowing that he preferred not to look at the detailed test cases, we decided to stick with an assertion format close to that of our existing tests.

Next, two people each tried a different set of tools for a proof of concept and shared their results with the rest of the team. It took a couple of rounds and a lot of time, but the investment paid off. The whole team chose the best tool set for our needs, and we brought in an expert coach to help us get a good start. The learning curve took a few weeks, but soon we were writing new tests quickly and enjoying a short feedback loop from maintainable tests.

Some people might think that taking time to experiment with how tests look, and the best framework to create them, is too expensive. But I can tell you from experience, it's much more expensive over the long term if you're trying to make do with the wrong test format and framework. Tests will fail more frequently, and every test failure will take longer to diagnose and correct. You will spend more and more time refactoring.

Lisa's experience illustrates the value of team members with different specialties and skill sets collaborating to find the most appropriate automation solutions for their team. One of the main proponents of the Page Object on her team was the lead system administrator, who also is a programmer and a proponent of useful test automation. His story follows.

Page Objects in Practice

Tony Sweets, an information technology architect from Colorado, USA, describes the advantages of using a Page Object pattern for automated tests through the UI. He has given us some code snippets and technical details, which we included in Appendix A, "Page Objects in Practice: Examples," for those readers who want more details. Full code for the example can be found on Tony's GitHub account (Sweets, 2013).

Object-oriented software is a time-tested way to produce maintainable, easy-to-modify code. Design patterns are key when writing in an object-oriented way; knowing what they are and how to use them will improve your code. A design pattern can be thought of as a recipe for how to build something. One pattern the software developer and testing community has accepted is the Page Object pattern that enables you to implement your automated tests in an object-oriented manner.

Understanding the Page Object Pattern

One of the problems that Page Objects try to solve is reuse. It is better to encapsulate the piece that you would have normally cut and pasted into a single module (or object) and reuse that single object where you need it. That way, you can generally just update the single object. As long as the interface to the object stays the same, the implementation details do not matter, and your tests should continue to work.

Your goal as an automated test writer is to separate your test from how it is implemented. For example, when you need to write a login test, the high-level test should not need to change when something in the application changes. Let's say this is your high-level test script:

Step 1: Open web browser and go to SUT site.

Step 2: Verify we are at the home page and there is a login box.

Step 3: Log in as Jane Doe.

Step 4: Verify that we are at Jane Doe's landing page.

Notice how I did not say how to log in. I did not say, “Type ‘jdoe’ in the username box and ‘password123’ in the password box, then hit Submit.” I simply said, “Log in as Jane Doe.” The thing or object that I’m calling should have those details. What if down the road a new requirement comes down from the product owner to add a third box to the login dialog for a PIN, such that you need a username, a PIN, and a password to log in? If you had 50 tests that started with logging in, you would have to make 50 changes. But if you’re using Page Objects, you have to make the change in only one place.

The idea is simple; you need a layer between your test and the implementation of the test. Your test can call methods on an object whose interface should be fairly stable. When changes happen in the system and you must account for them in your test (for example, there is a new requirement to add a CAPTCHA field on the login page, or the PIN in the preceding example), you can merely change the implementation of the login Page Object but not its interface; thus, you will not need to change your actual test.

Implementing the Page Object Pattern

The simple idea behind the Page Object pattern is to represent each page in your system as an object. This Page Object will encapsulate the elements of the page as well as any operations that can be done while viewing that page.

The first thing that a Page Object must do is verify that it is actually representing the page being displayed. This is usually done when the object is initialized. For example, a Home Page Object might verify that the words “Cool System Home Page” are in the title bar.

Operations (or methods) should return a Page Object. If, after logging in to an application, you expect to see a dashboard, your login method should return the dashboard Page Object.

There are no tests in the Page Object (other than verifying that the page has been rendered correctly) itself. Your tests are separate, and they are written like a script, but they do include these Page Objects and use them. The following script snippet shows the test script for verifying that a user can navigate to a specified page in the application under test:

```
// Goto the List of Nissan Cars
log.info("Clicking List of Nissan Cars");
NissanCarListPage nissanCarListPage =
nissanPage.selectListOfCars();
Assert.assertNotNull(nissanCarListPage);
```


The test framework (in this case, the `Assert` class of JUnit) lets you check for expected results such as nulls, equality, and true/false. A test failure stops the script and logs information about the failure so you can debug it in the test results via the build system, command line, or a report. At the end of the test script we send a quit command to the `WebDriver` object, which does cleanup such as closing down the browser.

See Appendix A, “Page Objects in Practice: Examples,” for Tony’s examples of how to implement the Page Object pattern, with code snippets and technical details. See the Part VI bibliography for more resources, including one from Anand Ramdeo on handling common problems with the Page Object model (Ramdeo, 2013).

TEST MAINTENANCE



As we noted earlier, you need to apply good design practices to create automated test code whose value exceeds its cost of maintenance. If you’re new to test automation, Dale Emery’s paper on how to write maintainable automated tests is a good place to start (Emery, 2009). Remember, this is an area where both programming and testing expertise is required, so collaborate and experiment to evolve test automation that works for your team.

Many teams automate and then forget about the tests, as long as they are passing. Over time, though, you may notice that the tests take longer and longer to run. Automated tests require continual attention. You may want to watch for duplication or places for refactoring because you find a better way. You may discover gaps in your automated test coverage. By giving visibility to your automated test results, you can continually monitor and ensure that they are still doing what you expect. Today’s test frameworks, whether you are testing at the service level or through the UI, can make writing automated tests fast and easy, so be aware that that isn’t the end of the story. You have to maintain those tests over time, make sure they return accurate results, and keep their feedback loop short. There are many areas to think about beyond writing the initial test.

The Importance of Test Data Management

Jeff “Cheezy” Morgan shares his story about the importance of managing test data for automation.

There is no worse feeling than coming into work in the morning, only to see that a large number of the tests that ran overnight have failed. As you investigate the reason for the failures, you quickly discover that somebody has changed the data that your tests depend on.

This scenario is all too common. Testers spend a lot of time investigating test failures, trying to discover if they failed due to a problem with the system under test or due to some environmental issue. When tests fail because of environmental issues, the team often loses faith in the tests. When that happens, they will likely start ignoring failures. Test data is a major contributor to environmental issues.

It is extremely important that our automated tests run consistently time and time again. In order to do this we have to have a good test data management strategy. Our tests can never fail because somebody has changed the data our tests depend on or because some data we were expecting to be in the database was not there.

The only way to ensure that the data is available in the exact state you want it to be is to have your tests create the data before each test runs. Also, in order to put the system back into a known state, your tests should clean up the data at the end of execution (see Figure 16-4).

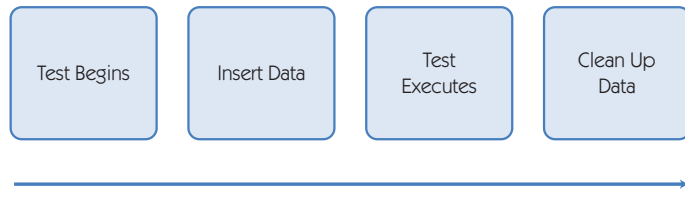


Figure 16-4 Data execution states

If your test suite has direct access to the database, this is easy. You can simply insert the data into the appropriate tables, execute your test, and then delete the data from the database.

Sometimes we do not have direct access to the database but instead must use some type of web service in order to set up the data. The

pattern should remain the same. We should use the services to create the data we need, execute the test, and then remove the data we used for our test.

What should we do if the data we are using comes from another company and we have no control over it? Further, what if that data is completely volatile? In cases like this we often need to build smart surrogate services that our test suites can run against in order to have consistent data. These services should allow the tests to specify the data to be returned when they are called under certain situations. These services introduce risk for our team, as we are not running against the actual services that our application will run against when in production. In order to mitigate this risk we need to test against the real services frequently.

If you are using the system under test to set up the test data you will use, you should consider using the Default Data pattern (Morgan, 2010) with randomized data. This will allow you to specify the data that matters while leaving the remainder of the data for the framework to create.

There are a few reasons why we remove the data after each test run. The simplest reason is that if we insert some data, modify it via a test, and then during a later test run attempt to insert the same data, we are likely to get an error. This error might be because some database tables have constraints that do not allow multiple copies of the same data.

Another reason to clean up the data with each test run is that data that sits in the database for long periods of time is suspect. We are never sure if somebody or something has changed the data since it was originally inserted. As a result, this data cannot be trusted.

Data management is an important part of test maintenance and often doesn't get the respect it deserves. There are libraries to generate randomized but realistic test data, such as Ruby Faker and jdefault (see the "Tools" section of the bibliography for links). Experiment with different options for creating and managing data for automated test to use. Make sure you have a way to reproduce any issues uncovered by automated tests using randomly generated data.

Another aspect of maintainability is ease of debugging test failures. It's important to be able to drill down into test failures to quickly identify exactly what went wrong. It can be helpful for test scripts to write

information to log files for more in-depth debugging when appropriate. Note that if you have kept your tests to single purpose, you may not need as much logging information, because it is simpler to find the failure.

SUMMARY

The same design principles, such as Don't Repeat Yourself (DRY), that help create robust production code also apply to test code. How we develop our tests will determine how much value we get out of them.

- Automated test code is as valuable as production code. It makes sense for the team's programmers to collaborate with testers to write that automated test code.
- In this chapter we discussed different design principles and patterns, including a three-level approach for UI automation and the Page Object pattern. Experiment with different principles and patterns to find what applies best to your tests.
- Understand how to collaborate to take advantage of tools that automate through the API/service level and beneath the UI layer.
- Data management for automated tests is critical, so spend time creating the right data strategy for your team or organization.
- Monitor the costs of maintaining automated regression tests, and take steps to keep them timely, with a positive return on investment.

This page intentionally left blank

SELECTING TEST AUTOMATION SOLUTIONS

17. Selecting Test Automation Solutions

Solutions for Teams in Transition

Meeting New Automation Challenges with the Whole Team

Achieving Team Consensus for Automation Solutions

How Much Automation Is Enough?

Collaborative Solutions for Choosing Tools

Scaling Automation to Large Organizations

Other Automation Considerations

You have some ideas about how to conquer your test technical debt. You've used the pyramid models from Chapter 15, "Pyramids of Automation," to consider a strategy for automating tests at different levels. You've used techniques from Chapter 16, "Test Automation Design Patterns and Approaches," to figure out what your tests at one or more levels should look like. Now you need to get down to specifics. For example, your team must build an infrastructure for writing and running automated tests, which might require new hardware and software. You have to select appropriate test drivers and frameworks, and in most cases this is a long-term effort. An incremental and iterative approach lets you apply what you learn along the way to grow valuable automated tests. In this chapter we will share not only our ideas, but stories from teams that have struggled through automation adoption and how they overcame their problems.

SOLUTIONS FOR TEAMS IN TRANSITION

Choosing and implementing new test automation solutions is especially challenging for teams that are also new to agile development. Getting a coach or having team members with experience in this area doesn't